

Digital clock progress log

Sara Tantarpale • Roll No. EE26BTECH11227

- Nethra Gupta • Roll No. EE26BTECH11219

Running the blink sketch on the arduino:

First, we tried to figure out how the arduino works, how the arduino IDE works and how to actually upload code into the arduino.



To make sure the arduino was working fine, we ran the blink sketch with the built in code in the arduino IDE (File => Examples=>0.1Basics=>Blink) which uses the Arduino's built in LED(which is connected to digital pin 13)

Issued wrong displays from the lab:

Coming to the rest of the components, it turns out we got common cathode displays instead of common anode displays from the lab , but the 74LS47 chip we had only works for common anode displays.

WHY?

From what we understand, it works like a light switch, but in reverse, in the sense it turns on by connecting to ground(0V) instead of sending out power(+5V).

- **Normal (Active-HIGH) logic:**

The pin of the chip outputs(+5V) to turn an LED on, and turns it off by connecting to ground(sending 0V).

- Meanwhile for the 74LS47:

The pin turns an LED on by pulling the signal down to 0V and when it wants the LED off, it sits at high voltage.

How it works:

[Active - LOW: to turn a specific segment on, the chip connects to ground(0V).

Active - HIGH: to turn a specific segment on, the chip sends out power(+5V).]

- In common anode displays, all the LEDs are connected to high voltage(+5V) on one side(from the COM pin), for current to flow, the required voltage difference is provided by the chip, when the pin drops to 0V, electricity flows from the display down into the chip and it lights up.
- This is why the common cathode wouldn't have worked since all the LEDs are connected to ground(0V) on one side, they require high voltage to light up which an active- LOW pin would never send out.

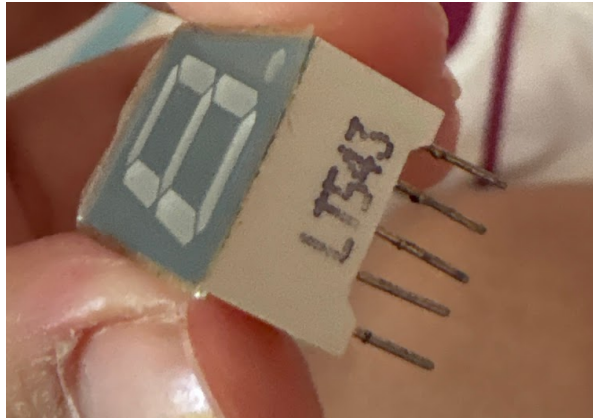
How we fixed it :

We could have either used a 74LS48 chip which works on active- HIGH logic or replace all the segments with common anode, which is what we did.

How to distinguish between common anode and common cathode displays:(to make sure we didn't get scammed when issuing the displays from the lab).



- This one is a common cathode, the 5611 is a standard series where: 56 denotes 0.56 inch display the first “1” denotes that it is a single digit display(5621 would be a 2 digit display, 5641 would be a 4 digit display..etc) and the second “1” denotes the exact pin assignment scheme, i.e., in this case, this display is a 10 pin single digit display with common pins on pin 3 and 8.
- The C indicates common cathode.(5611A would have denoted common anode)
(can also be represented as 5611AH which denotes common cathode,
5611BH denotes common cathode)

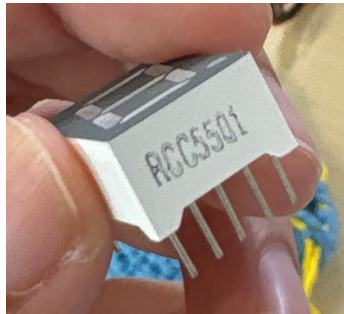


- LT543 indicates a common cathode display whereas LT542 or LT544 would have indicated common anode:
- Here 54 again denotes the digit height(0.52 to 0.56 inch display) and 3 denotes the specific 10 pin layout.



- This one was also common cathode,5503 indicates common cathode, whereas common anode would show 5501.

- FINALLY, rechecked and replaced all the common cathode displays with common anode and double checked(5501 = common anode) according to the standard convention of 5500 series.



Single digit seconds clock:

To get the hang of how the clock and code works, (without multiplexing) and to check if all the displays were working we simply connected each display one by one to see if it was lighting up upon uploading the code to run the numbers from 0 to 9.

Since we weren't multiplexing, we didn't require the chip for this either.

[View here](#)

The blueprint we followed:

Connected all the segments individually to the arduino (no multiplexing):

(COM connected to 0V)

7 segment display:

a segment - D2 of arduino
b segment - D3 of arduino
c segment - D4 of arduino
d segment - D5 of arduino
e segment - D6 of arduino
f segment - D7 of arduino
g segment - D8 of arduino

How the code works:

- pinMode sets each individual segment to output

- Switch function works as a neater version of a long chain of if- else loops
(switch(variable): the function checks if the variable matches each individual case, if it does, it breaks, if not it moves on to the next case.)
- digitalWrite(pin, HIGH/LOW) command sends HIGH(5V) or LOW(0V) signal to the specific pin on the arduino. For this common anode single digit clock, LOW lights up the segment and HIGH turns it off, whereas for common cathode it would be reversed.
- delay(time) function runs each iteration after the given time(in milliseconds) specified in the function.
- num ++ adds 1 after each iteration.
- The last if statement is to reset num value to 0 after 9

Here ON = LOW output and OFF = HIGH output

For 0 : segments 2, 3, 4, 5, 6, 7 have to be ON

For 1: segments 3, 4 have to be ON

For 2: segments 2, 3, 5, 6, 8 have to be ON

For 3: segments 2, 3, 4, 5, 8 have to be ON

For 4: segments 3, 4, 7, 8 have to be ON

For 5: segments 2, 4, 5, 7, 8 have to be ON

For 6: segments 2 4 5 6 7 8 have to be ON

For 7: segments 2 3 4 have to be ON

For 8: segments 2 3 4 5 6 7 8 have to be ON

For 9: segments 2 3 4 5 7 8 have to be ON

Code:

```
int num = 0;
```

```
void setup() {
  pinMode(2, OUTPUT);
  pinMode(3, OUTPUT);
  pinMode(4, OUTPUT);
  pinMode(5, OUTPUT);
  pinMode(6, OUTPUT);
  pinMode(7, OUTPUT);
  pinMode(8, OUTPUT);
}
```

```
void loop() {
  // Display the current number
```

```
switch(num) {
```

case 0:

```
digitalWrite(2, LOW);  
digitalWrite(3, LOW);  
digitalWrite(4, LOW);  
digitalWrite(5, LOW);  
digitalWrite(6, LOW);  
digitalWrite(7, LOW);  
digitalWrite(8, HIGH);  
break;
```

case 1:

```
digitalWrite(2, HIGH);  
digitalWrite(3, LOW);  
digitalWrite(4, LOW);  
digitalWrite(5, HIGH);  
digitalWrite(6, HIGH);  
digitalWrite(7, HIGH);  
digitalWrite(8, HIGH);  
break;
```

case 2:

```
digitalWrite(2, LOW);  
digitalWrite(3, LOW);  
digitalWrite(4, HIGH);  
digitalWrite(5, LOW);  
digitalWrite(6, LOW);  
digitalWrite(7, HIGH);  
digitalWrite(8, LOW);  
break;
```

case 3:

```
digitalWrite(2, LOW);
```

```
digitalWrite(3, LOW);  
digitalWrite(4, LOW);  
digitalWrite(5, LOW);  
digitalWrite(6, HIGH);  
digitalWrite(7, HIGH);  
digitalWrite(8, LOW);  
break;
```

case 4:

```
digitalWrite(2, HIGH);  
digitalWrite(3, LOW);  
digitalWrite(4, LOW);  
digitalWrite(5, HIGH);  
digitalWrite(6, HIGH);  
digitalWrite(7, LOW);  
digitalWrite(8, LOW);  
break;
```

case 5:

```
digitalWrite(2, LOW);  
digitalWrite(3, HIGH);  
digitalWrite(4, LOW);  
digitalWrite(5, LOW);  
digitalWrite(6, HIGH);  
digitalWrite(7, LOW);  
digitalWrite(8, LOW);  
break;
```

case 6:

```
digitalWrite(2, LOW);  
digitalWrite(3, HIGH);  
digitalWrite(4, LOW);  
digitalWrite(5, LOW);
```

```
digitalWrite(6, LOW);  
digitalWrite(7, LOW);  
digitalWrite(8, LOW);  
break;
```

case 7:

```
digitalWrite(2, LOW);  
digitalWrite(3, LOW);  
digitalWrite(4, LOW);  
digitalWrite(5, HIGH);  
digitalWrite(6, HIGH);  
digitalWrite(7, HIGH);  
digitalWrite(8, HIGH);  
break;
```

case 8:

```
digitalWrite(2, LOW);  
digitalWrite(3, LOW);  
digitalWrite(4, LOW);  
digitalWrite(5, LOW);  
digitalWrite(6, LOW);  
digitalWrite(7, LOW);  
digitalWrite(8, LOW);  
break;
```

case 9:

```
digitalWrite(2, LOW);  
digitalWrite(3, LOW);  
digitalWrite(4, LOW);  
digitalWrite(5, LOW);  
digitalWrite(6, HIGH);  
digitalWrite(7, LOW);  
digitalWrite(8, LOW);
```

```
break;
}

delay(1000); // Wait 1 second before moving to the next number

num++;
if (num > 9) {
    num = 0; // Reset back to 0 after reaching 9
}
}
```

Note :

For the main code we used to run the digital clock, we figured out how to use the write(BCD) function so we wouldn't have to write all 50 digitalWrite statements.

FOR THE ACTUAL PROJECT:

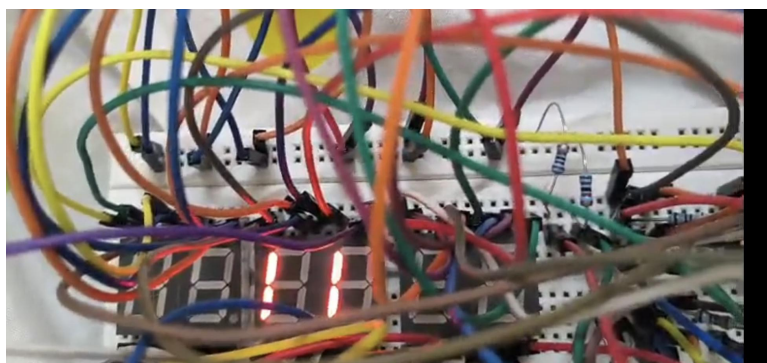
(adding multiplexing)

We started looking for a YouTube tutorial for a six-digit digital clock with individual displays , but finding one that actually matched what we were doing was surprisingly difficult. Most of the videos either used only four displays or already had the displays connected, so we couldn't really follow them . We found a 10 hour YouTube video that explained how the displays, arduino and code worked. We didn't exactly sit through the entire 10 hours, but the video helped us understand the basic idea of multiplexing and gave us an idea of how we could connect and control all six displays together.

Starting from ground 0:

Initially we didn't follow the reference pdf at all, we just directly asked AI what to connect and ended up following a totally different set of pin connections that just didn't end up working.

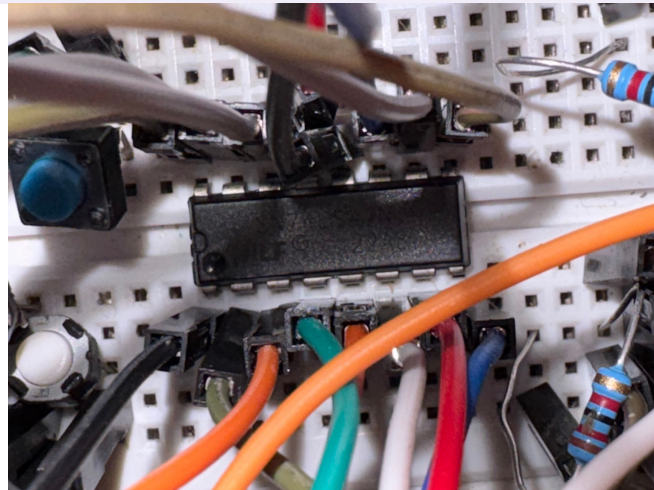
The first time we connected everything, it turns out, we hadn't even used the **7447** at all because according to the instructions given by AI, it wasn't necessary. (What were we even thinking??!)



(No chip) and as is clearly visible, horribly malfunctioning.

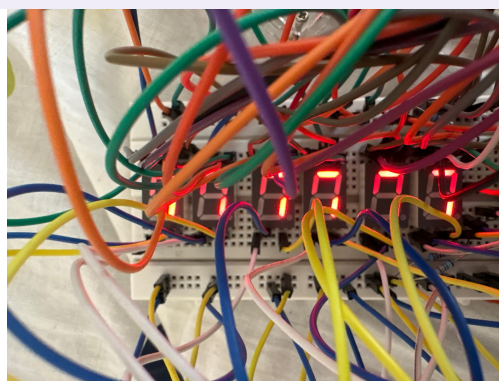
Luckily we realised soon enough that the chip actually was required so we went back, connected all the required wires to the chip, and tried again. And once again, it didn't work.

Faulty chip?



We rechecked all the wire connections and after checking everything for what felt like the hundredth time, we realised the **7447 was actually inverted**. (Rookie mistake) . Turns out we took pin 16 as pin 1.

More malfunctioning:



We inverted the chip and then decided on the final blueprint.

The final blueprint:

After a lot of research, and consulting the reference pdf , (modifying it for common anode instead of cathode), we finally decided upon the following blueprint to build the clock(which actually did end up working):

The components we used:

- Arduino uno

- 74LS47 chip
- 240 ohm resistors
- Jumper wires
- 6 7 segment common anode displays.
- 3 push buttons

Our pin connections:

All the various segments, connected in parallel to a 240 ohm resistor and then connected to pins on the 74LS47 IC.

a segments(pin 7 on display) - 240 ohm resistor - pin 13 on the 74LS47

b segments(pin 6 on display) - 240 ohm resistor - pin 12 on the 74LS47

c segments (pin 4 on display)- 240 ohm resistor - pin 11 on the 74LS47

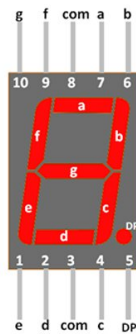
d segments (pin 2 on display)- 240 ohm resistor - pin 10 on the 74LS47

e segments (pin 1 on display) - 240 ohm resistor - pin 9 on the 74LS47

f segments (pin 9 on display)- 240 ohm resistor - pin 15 on the 74LS47

g segments(pin 10 on display) - 240 ohm resistor - pin 14 on the 74LS47

(3 and 8 are the COM pins and 5 is the dot)



Pin 16 of the IC is connected to 5V on the arduino

Pin 8 of the IC connected to ground on the arduino

Arduino input pins:

(Connecting arduino digital pins to BCD pins on the IC 7447)

Pin 7 of the IC (BCD Bit A) - connected to the D0 digital pin of the arduino

Pin 1 of the IC (BCD Bit B) - connected to the D1 digital pin of the arduino

Pin 2 of the IC (BCD Bit C) - connected to the D2 digital pin of the arduino

Pin 6 of the IC (BCD Bit D) - connected to the D3 digital pin of the arduino

Digit Enabling lines (Multiplexing):

Display 1 (Hours Tens) - D4 on Arduino

Display 2 (Hours Units) - D5 on Arduino

Display 3 (Minutes Tens) - D6 on Arduino

Display 4 (Minutes Units) - D7 on Arduino

Display 5 (Seconds Tens) - D8 on Arduino

Display 6 (Seconds Units) - D9 on Arduino.

Hours push button - D10 on Arduino

Minutes push button - D9 on Arduino

Putting the circuit together:

We finally started building the actual circuit. We first tried connecting all the displays together (daisy wiring) , but halfway through we realised that there just weren't enough holes on the breadboard to connect all the pins directly. After trying to figure out a solution for a long time, we realised we could use the long rails on the breadboard for some connections and the middle parts for the rest.

But even after doing all that, some of the display segments still weren't working. We spent almost an entire day checking wires and connections and trying to figure out what was wrong. At one point, we were so convinced that the breadboard itself was faulty that we decided we were just going to get it exchanged from Core Labs.

Then, out of desperation we decided to take one last look at the breadboard and removed the double-sided tape underneath it.

And that's when we made a “groundbreaking” discovery.

Turns out that all the rails are not connected. The rod only connects the rails till the middle part and not the entire rail.

Then we decided to connect the rails using jumper wires and uploaded the code again.

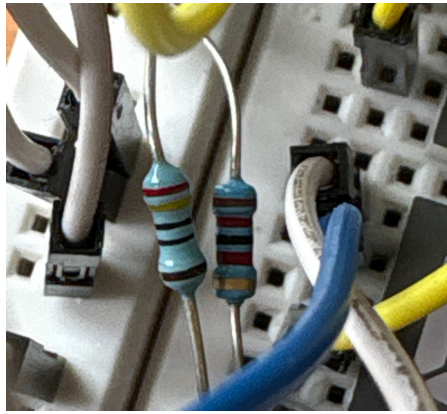
After doing that, the clock finally started to *almost* work. Some of the segments were lighting up, but a few still weren't. At this point, we couldn't even tell if the problem was the code or just another loose connection.

So we started messing around with the connections, and we noticed that whenever we touched one of the resistors, some of the segments would suddenly light up or stop working.

After closely observing the resistors we realised another problem.

We had resistors with different resistance values, 4.7 kOhms and 10 kOhms, when we actually needed 220 Ohms resistors.

Rookie mistake, we forgot to check all the resistors, the one on the left is 240 ohms but the one on the left is 10 kilo ohms and neither of us spotted it.



Somehow, we managed to get the correct resistors and replaced all of them. At this point, we thought that we were really close. Except we weren't. When the breadboard was kept flat, random segments started lighting up.

The breadboard literally had to be kept at a certain angle for some of the segments to work. If we moved it even slightly, they would stop working.

Then we started panicking because how are we supposed to explain in a demo, that you have to hold the clock at a 45 degree angle for it to work ?

[View here](#)

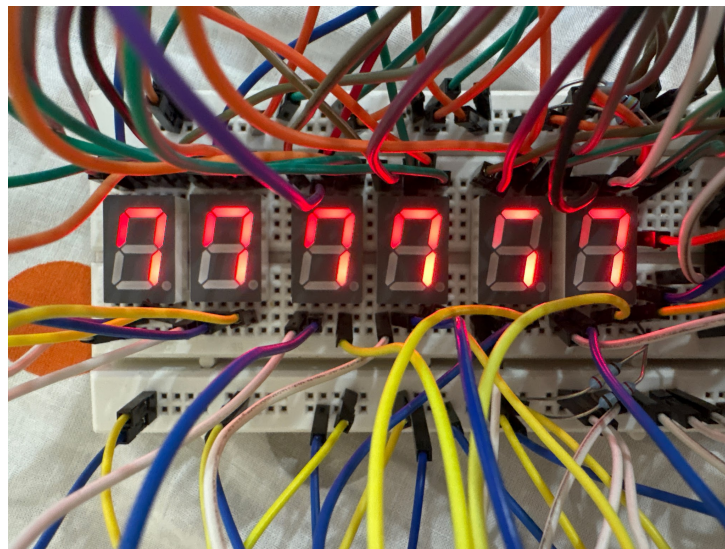
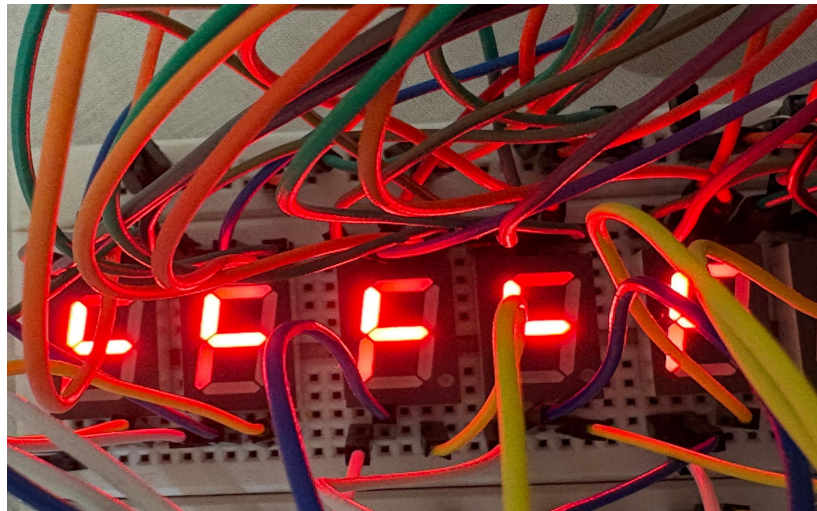
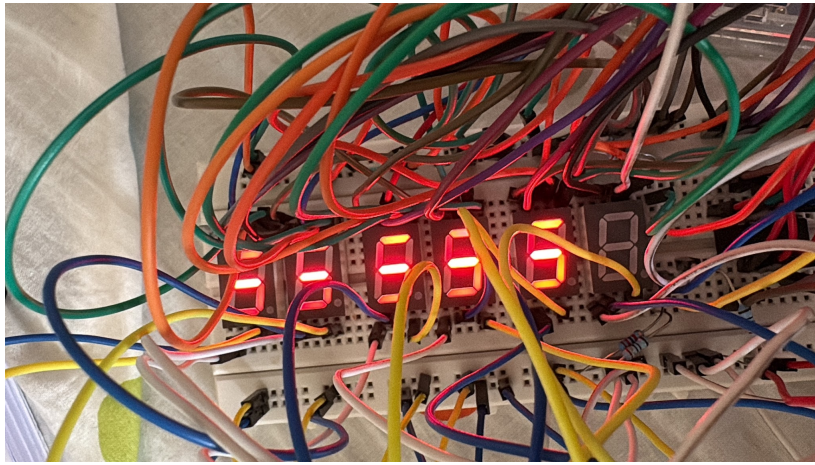
Just lighting up displays first:

We realised trying to set up the whole clock along with push buttons in one go was being a little too ambitious, so we decided to just light up all the displays, without the push buttons or code first to make sure everything was working.

Even that proved to be a challenge.

SO MUCH MALFUNCTIONING:

It was either loose wires, resistors, accidental wrong pin connections, in one instance we forgot to connect the IC pin 8 to ground, all of which led to random segments lighting up.



Our clock had been malfunctioning quite badly, with random segments lighting up even when they were not supposed to. After spending almost three days rechecking the circuit, rewiring everything, and trying to figure out what was causing the problem, we finally found one of the major issues.

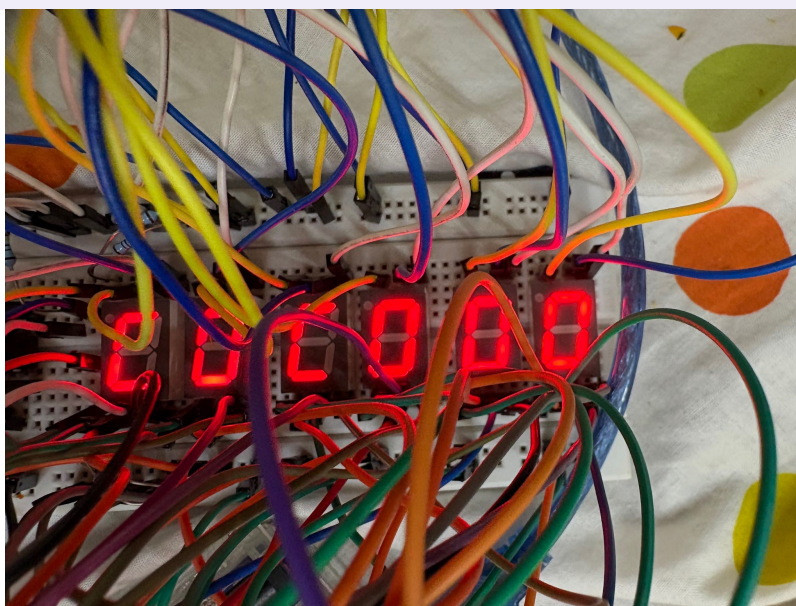
Pin 8 of one of the 74LS47 ICs was not connected to GND at all. It was simply left unconnected.

Then one morning, while checking all the wires and connections for what felt like the millionth time, we found another issue. One of the resistors that was supposed to be connected was actually not connected because the wire had been placed in a different rail.

And then, because apparently there had to be one more problem, we noticed that some of the resistors were touching each other as well. So, once again, we went back to carefully checking every single connection. At this point, we've realised that with this project, even the smallest wiring mistake can cause the entire circuit to behave randomly.

The problem of the random displays lighting up we figured was due to mismatched pins, and instead of rechecking everything, we just decided to take it apart and rewire everything and that fixed the problem for the most part, in the sense the clock was functioning, but none of the g segments were lighting up.

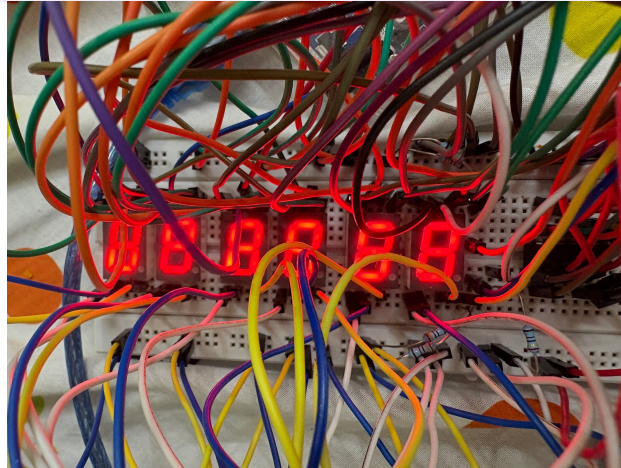
None of the g segments lighting up:



We tried everything, first we removed the wire connecting the g segments to the chip and connected it to the ground pin of the arduino to check if the connections were correct.

And they were, when we connected it, all the g segments lit up. Then we thought it was a chip problem, so we replaced the chip but the same 14th pin on the chip still wasn't providing any current. Then we finally figured out it was a breadboard problem and the rail connecting the 14th pin of the chip was faulty, so we moved everything a couple of rails down and all the displays lit up! Yay!

FIXED THE DISPLAYS!



Next we uploaded the clock code, but the displays were still showing random segments and not proper numbers.

[View here](#)

We figured it was wrong pin wiring again because all the different codes we used were lighting up the same pattern of “wrong segments”, so couldn't be a problem with the individual wires.

We decided to just take it apart and put it together for a third time, then FINALLY it started working.

CLOCK FINALLY WORKING:

We first tried to run it without push buttons, after 20 tries, that worked:

[View here](#)

Added push buttons:

Next we added the push buttons for hours and minutes, after a little toggling with the buttons and wires, we got that to work as well.

First we connected one leg of the button to the 5V of the arduino and the other leg to the D10 pin of the arduino. Then this 2nd leg was also connected to a 10kOhms resistor which was further connected to the GND pin of the arduino. This button incremented the digits on the hour (HH) display. Similarly we added buttons for incrementing the minutes as well.

The clock was still malfunctioning with the push buttons but it was only a problem of loose connections, which we ended up fixing.

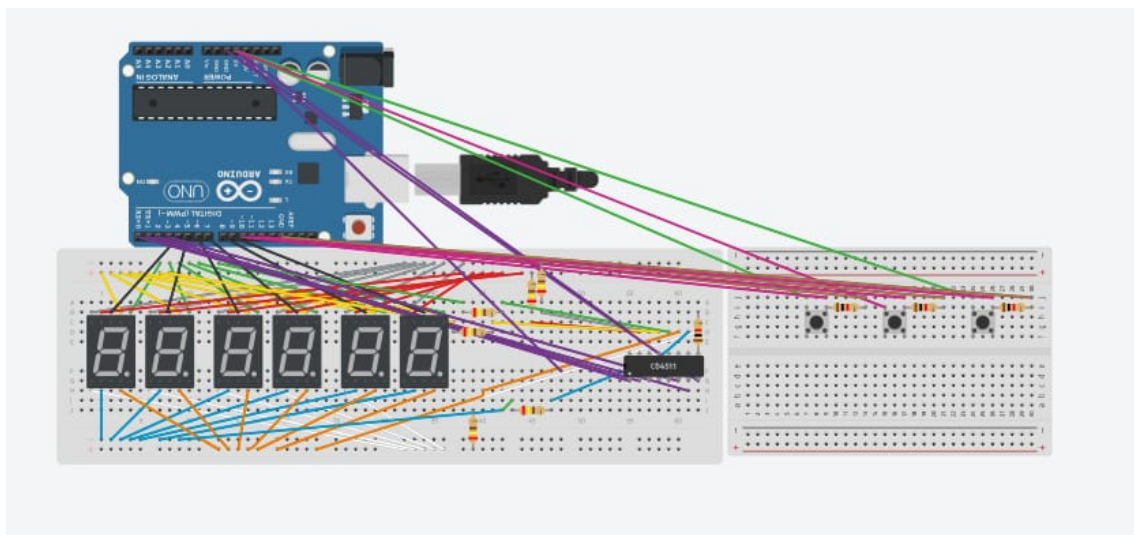
[View here](#)

EXTRA STOPWATCH FEATURE:

Next we decided to add a stopwatch feature to the clock. For this we needed to add another button to switch from 24 hr clock mode to stopwatch mode. After adding the button and connecting it to the D12 pin, we tweaked the code a little bit to switch between both the modes.

[View here](#)

TINKERCAD SIMULATION:



Understanding the code:

Final code we used:

```
#include <Arduino.h>

// =====

// 6 DIGIT DIGITAL CLOCK + STOPWATCH

// Arduino UNO + 74LS47

// =====
```

```
// ----- DISPLAY CONNECTIONS -----
```

```
// 7447 BCD input pins
```

- `const int` declares an integer variable, and “`const`” makes sure that the value of the integer cannot be changed in later parts of the code at any point of time.
- `const int xyzPins[L] = {0, 1, 2, etc..}` creates an array of length L holding the digital arduino pins mentioned:

```
const int bcdPins[4] = {0, 1, 2, 3};
```

```
// Six display enable pins
```

```
const int digitPins[6] = {4, 5, 6, 7, 8, 9};
```

```
// ----- BUTTON CONNECTIONS -----
```

```
// Button 1: Hour in clock / Start-Stop in stopwatch
```

- Similarly, the hour button is assigned to arduino pin 10 minute button to pin 11 and the mode button(to switch from 24 hour clock mode to stopwatch mode, is assigned to pin 12

```
const int hourButton = 10;
```

```
// Button 2: Minute in clock / Reset in stopwatch
```

```
const int minuteButton = 11;
```

```
// Button 3: Clock <-> Stopwatch
```

```
const int modeButton = 12;
```

```
// ----- CLOCK VARIABLES -----
```

- Next, variables are initialized, hours, minutes and seconds variables are assigned values 12, 00 and 00 respectively to make sure the clock starts at 12:00:00.

```
int hours = 12;
```

```
int minutes = 0;
```

```
int seconds = 0;
```

- `Unsigned long` is another integer storing function that stores only positive integers.
Stores the timestamp (in milliseconds) of when the clock last ticked by 1 second.

```
unsigned long previousClockMillis = 0;
```

```
// ----- STOPWATCH VARIABLES -----
```

- **A True/False flag tracking which mode is being operated:**

false = Clock Mode, true = Stopwatch Mode.

```
bool stopwatchMode = false;
```

- If the stopwatch is actively counting up , this boolean will show true, if it is paused, itll show false.

```
bool stopwatchRunning = false;
```

- Stores the exact timestamp when the stopwatch was started/resumed.

```
unsigned long stopwatchStartMillis = 0;
```

- stores total elapsed time in milliseconds so time isn't lost when pausing/resuming.

```
unsigned long stopwatchElapsed = 0;
```

```
// ----- BUTTON VARIABLES -----
```

```
bool lastHourButtonState = LOW;
```

```
bool lastMinuteButtonState = LOW;
```

```
bool lastModeButtonState = LOW;
```

```
unsigned long lastHourPress = 0;
```

```
unsigned long lastMinutePress = 0;
```

```
unsigned long lastModePress = 0;
```

```
const unsigned long debounceTime = 200;
```

```
// =====
```

```
// SETUP
```

```
// =====
```

- Code in the setup function runs only once

```
void setup() {
```

```
// 7447 BCD pins
```

- Sets pins D0, D1, D2, and D3 as outputs to send data to the 7447 decoder.

```
for (int i = 0; i < 4; i++) {
    pinMode(bcdPins[i], OUTPUT);
}
```

- Sets pins D4 through D9 as outputs and immediately sets them LOW (all displays initially OFF).

```
// Display enable pins
for (int i = 0; i < 6; i++) {
    pinMode(digitPins[i], OUTPUT);
    digitalWrite(digitPins[i], LOW);
}
```

```
// Buttons
```

```
// Your wiring uses external 10k pull-down resistors.
```

- Sets Pins 10, 11, and 12 as inputs for reading button presses

```
pinMode(hourButton, INPUT);
pinMode(minuteButton, INPUT);
pinMode(modeButton, INPUT);
}
```

```
// =====
```

```
// WRITE BCD TO 7447
```

```
// =====
```

Takes a single number `val` (0–9) and outputs its 4-bit binary equivalent across pins D0–D3.

- `(val>>i)` shifts the binary number to inspect bit `i`.
- `& 0x01` isolates that single bit (0 or 1).
- `digitalWrite` sends the voltage to the 7447 input pin.

If we were to write this part of the code manually with individual `digitalWrite` functions,

To send the number 9 (which is 1001 in binary), you have to set Pins 0, 1, 2, and 3 individually:

```
digitalWrite(0, HIGH); Bit 0 (1) -> ON
digitalWrite(1, LOW); Bit 1 (0) -> OFF
digitalWrite(2, LOW); Bit 2 (0) -> OFF
digitalWrite(3, HIGH); Bit 3 (1) -> ON
```

And we would have to write at least 40 lines of code

```
void writeBCD(int val) {
```

```
    for (int i = 0; i < 4; i++) {
```

```
        digitalWrite(bcdPins[i], (val >> i) & 0x01);
```

```
    }
```

```
}
```

```
// =====
```

```
// DISPLAY SIX DIGITS
```

```
// =====
```

- Takes an array of 6 numbers and turns each display on and off sequentially for 1 ms. Because this repeats hundreds of times per second, your eyes see all 6 digits lit simultaneously.

```
void displayDigits(int digits[6]) {
```

```
    for (int i = 0; i < 6; i++) {
```

```
        // Send BCD value
```

```
        writeBCD(digits[i]);
```

```
        // Turn display ON
```

```
        digitalWrite(digitPins[i], HIGH);
```

```
        // Short multiplexing delay
```

```
        delay(1);
```

```
        // Turn display OFF
```

```
        digitalWrite(digitPins[i], LOW);
```

```
    }
```

```
}
```

```
// =====
```

```
// DISPLAY CLOCK
```

```
// =====
```

- Splits hours, minutes, and seconds into 6 separate digits and sends them to the display function.

```
void displayClock() {
```

```
    int digits[6];
```

```
    digits[0] = hours / 10;
```

```
    digits[1] = hours % 10;
```

```
    digits[2] = minutes / 10;
```

```
    digits[3] = minutes % 10;
```

```
    digits[4] = seconds / 10;
```

```
    digits[5] = seconds % 10;
```

```
    displayDigits(digits);
```

```
}
```

```
// =====
```

```
// DISPLAY STOPWATCH
```

```
// Format: MM:SS:CC
```

```
// CC = centiseconds (1/100 second)
```

```
// =====
```

```
void displayStopwatch() {
```

```
    unsigned long elapsed = stopwatchElapsed;
```

```
    // If stopwatch is running, calculate current elapsed time
```

```
    if (stopwatchRunning) {
```

```
        elapsed = millis() - stopwatchStartMillis;
```

```
}
```

- Converts total elapsed time into minutes, seconds and centiseconds.

```
unsigned long totalCentiseconds = elapsed / 10;
```

```
int stopwatchMinutes =
```

```
(totalCentiseconds / 6000) % 100;
```

```
int stopwatchSeconds =
```

```
(totalCentiseconds / 100) % 60;
```

```
int centiseconds =
```

```
totalCentiseconds % 100;
```

- Splits stopwatch values into 6 individual digits (MM:SS:CC) and sends them to displayDigits()

```
int digits[6];
```

```
digits[0] = stopwatchMinutes / 10;
```

```
digits[1] = stopwatchMinutes % 10;
```

```
digits[2] = stopwatchSeconds / 10;
```

```
digits[3] = stopwatchSeconds % 10;
```

```
digits[4] = centiseconds / 10;
```

```
digits[5] = centiseconds % 10;
```

```
displayDigits(digits);
```

```
}
```

```
// =====
```

```
// UPDATE NORMAL CLOCK
```

```
// =====
```

- Every 1 second, updateclock function updates the seconds variable.

```
void updateClock() {
```

```

unsigned long currentMillis = millis();

if (currentMillis - previousClockMillis >= 1000) {

    previousClockMillis += 1000;

    seconds++;

    if (seconds >= 60) {
        seconds = 0;
        minutes++;

        if (minutes >= 60) {
            minutes = 0;
            hours++;

            if (hours >= 24) {
                hours = 0;
            }
        }
    }
}

// =====
// HANDLE HOUR BUTTON
// =====

    • Detects when the hour button transitions from LOW to HIGH.
    • Clock mode: changes hours.
    • Stopwatch mode: toggles between start and stop.

void handleHourButton() {

    bool currentState = digitalRead(hourButton);

```

```
// Detect LOW -> HIGH transition
if (currentState == HIGH &&
    lastHourButtonState == LOW &&
    millis() - lastHourPress > debounceTime) {

    lastHourPress = millis();

    if (!stopwatchMode) {
```

// CLOCK MODE:

```
// Increase hour
hours++;

if (hours >= 24) {
    hours = 0;
}

} else {
```

// STOPWATCH MODE:

```
// Start / Stop

if (!stopwatchRunning) {

    // Start/resume stopwatch
    stopwatchStartMillis =
        millis() - stopwatchElapsed;

    stopwatchRunning = true;

} else {

    // Stop stopwatch
```

```

    stopwatchElapsed =
        millis() - stopwatchStartMillis;

    stopwatchRunning = false;
}
}
}

lastHourButtonState = currentState;
}

// =====
// HANDLE MINUTE BUTTON
// =====

void handleMinuteButton() {

    bool currentState = digitalRead(minuteButton);

    // Detect LOW -> HIGH transition
    if (currentState == HIGH &&
        lastMinuteButtonState == LOW &&
        millis() - lastMinutePress > debounceTime) {

        lastMinutePress = millis();

        if (!stopwatchMode) {

```

// CLOCK MODE:

```

// Increase minute

minutes++;

if (minutes >= 60) {

```

```
    minutes = 0;
}
```

```
} else {
```

// STOPWATCH MODE:

```
// Reset stopwatch
```

```
stopwatchRunning = false;
stopwatchElapsed = 0;
stopwatchStartMillis = millis();
```

```
}
```

```
}
```

```
lastMinuteButtonState = currentState;
```

```
}
```

```
// =====
```

```
// HANDLE MODE BUTTON
```

```
// =====
```

```
void handleModeButton() {
```

```
    bool currentState = digitalRead(modeButton);
```

```
    // Detect LOW -> HIGH transition
```

```
    if (currentState == HIGH &&
```

```
        lastModeButtonState == LOW &&
```

```
        millis() - lastModePress > debounceTime) {
```

- The millis() function counts the number of milliseconds that have passed since the Arduino has been powered on.

```
    lastModePress = millis();
```

```

// Toggle mode
stopwatchMode = !stopwatchMode;

// If returning to clock mode,
// stop stopwatch but keep its value.
if (!stopwatchMode) {

    if (stopwatchRunning) {
        stopwatchElapsed =
            millis() - stopwatchStartMillis;

        stopwatchRunning = false;
    }
}

lastModeButtonState = currentState;
}

// =====
// MAIN LOOP
// =====

void loop() {

    // Keep normal clock running in the background
    updateClock();

    // Check all buttons
    handleHourButton();
    handleMinuteButton();
    handleModeButton();

```

```
// Display whichever mode we're currently in
if (stopwatchMode) {
    displayStopwatch();
} else {
    displayClock();
}
}
```